Invertebrate Species Database for the Cunha Lab

Advanced User Guide

This guide is intended for advanced users and lab members with more programming experience who are responsible for adding new datasets, modifying the schema, maintaining scripts, and managing the database long-term. This guide covers command-line database access, the full script pipeline, onboarding new field site datasets, schema changes, deleting data, advanced SQL, database maintenance, and extending the validation system.

For information on basic SQL queries, adding single rows, renaming columns, or GUI-based edits, see the `Beginner User Guide` for those topics.

Command-Line Database Access

Opening the database in the SQLite 3 shell

The `sqlite3` CLI lets you run SQL directly against the database file without any Python or GUI involved. This is useful for quick checks, one-off edits, and debugging.

```
sqlite3 initial-building-scripts/cunha_invertebrate_specimens.db
```

You will see a `sqlite>` prompt. Some useful shell commands:

Command	What it does
`.tables`	List all tables
`.schema TableName`	Show the CREATE TABLE statement for a table
`.headers on`	Show column names in query output
`.mode column`	Format output as aligned columns
`.mode csv`	Format output as CSV
`.output results.csv`	Redirect all output to a file
`.output stdout`	Return output to the terminal

Command	What it does
`.quit`	Exit the shell

Running a Python script against the database

All Python scripts in this repo use `sqlite3.connect()` to open a connection to the database. The general pattern used throughout the codebase is:

```
import sqlite3

import pandas as pd



conn = sqlite3.connect("initial-building-scripts/cunha_invertebrate_specimens.db")

cur = conn.cursor()



# Always enable foreign key enforcement as it is OFF by default in SQLite and does not persist between connections

cur.execute("PRAGMA foreign_keys = ON;")



# ... cleaning, querying, etc ...



conn.commit()   # write changes permanently

conn.close()    # release the file lock
```

> ⚠ **Important**

`PRAGMA foreign_keys = ON` must be run at the start of every connection. SQLite does not store this setting in the file and it resets to OFF each time a connection is opened. OFF means foreign key constraints will not be enforced during that session.

Database Schema Reference

Table relationships

```
EventData

    └── SpecimenData        (event_code → EventData.event_code)

            └── DNAExtractions      (lot_id → SpecimenData.lot_id)

                    └── GenomicLibraries    (extraction_id →
DNAExtractions.extraction_id)
```

You must load parent rows before child rows, and you cannot delete a parent row that has children referencing it when foreign keys are ON.

EventData

Column	Type	Notes
`event_code`	TEXT	**PRIMARY KEY**
`trip_id`	TEXT	
`year`	INTEGER	
`month`	INTEGER	
`day`	INTEGER	
`date`	TEXT	
`waypoint`	TEXT	
`latitude`	REAL	

Column	Type	Notes
`longitude`	REAL	
`environment`	TEXT	
`collecting_method`	TEXT	
`depth`	TEXT	
`locality`	TEXT	
`locality_details`	TEXT	Added with La Palma
`city_district`	TEXT	Added with La Palma
`province`	TEXT	
`area`	TEXT	
`country`	TEXT	
`collector`	TEXT	
`event_notes`	TEXT	

SpecimenData

Column	Type	Notes
`lot_id`	TEXT	**PRIMARY KEY**
`suffix`	TEXT	
`event_code`	TEXT	**FOREIGN KEY** → EventData
`species`	TEXT	
`genus`	TEXT	
`development`	TEXT	
`habitat`	TEXT	
`fixation_method`	TEXT	
`specimen_count`	INTEGER	
`parts`	TEXT	
`specimen_notes`	TEXT	
`identification_by`	TEXT	
`voucher`	TEXT	
`second_voucher_clip`	TEXT	

Column	Type	Notes
`dna_extracted`	TEXT	Added with La Palma
`phylum`	TEXT	Added with La Palma
`class_name`	TEXT	
`subclass`	TEXT	Added with La Palma
`uncertainty`	TEXT	Added with La Palma
`collected_by`	TEXT	Added with La Palma
`museum`	TEXT	Added via `add_cols_sql.py`
`specimenlocation_id`	TEXT	Added via `add_cols_sql.py`
`specimenlocation_shelf`	TEXT	Added via `add_cols_sql.py`

DNAExtractions

Column	Type	Notes
`extraction_id`	TEXT	**PRIMARY KEY** (La Palma conflicts prefixed with `LP-`)
`lot_id`	TEXT	**FOREIGN KEY** → SpecimenData
`extraction_date`	TEXT	
`extraction_kit`	TEXT	
`elution_ul`	REAL	
`qubit_dna_ng_ul`	REAL	
`nanodrop_ng_ul`	REAL	NULL for La Palma
`nanodrop_260_280`	REAL	NULL for La Palma
`nanodrop_260_230`	REAL	NULL for La Palma
`qubit_nanodrop_ratio`	REAL	NULL for La Palma
`clip_over`	TEXT	NULL for La Palma
`contamination_plate`	TEXT	NULL for La Palma
`contamination_wells`	TEXT	NULL for La Palma
`extraction_notes`	TEXT	
`piece_size`	TEXT	
`qubit_after_speedvac`	REAL	NULL for La Palma

Column	Type	Notes
`extraction_suffix`	TEXT	La Palma only
`clipping_notes`	TEXT	La Palma only
`extracted_by`	TEXT	La Palma only
`coi`	TEXT	La Palma only
`pcr_by`	TEXT	La Palma only
`blast_best_match`	TEXT	La Palma only
`blast_max_score`	REAL	La Palma only
`blast_total_score`	REAL	La Palma only
`blast_query_cover`	TEXT	La Palma only
`blast_e_value`	TEXT	La Palma only
`blast_percent_identity`	TEXT	La Palma only
`blast_match_length`	INTEGER	La Palma only
`blast_match_accession`	TEXT	La Palma only
`original_extraction_id`	TEXT	Stores original ID of La Palma conflicts before `LP-` prefix was applied
`dnalocation_id`	TEXT	Added via `add_cols_sql.py`
`dnalocation_shelf`	TEXT	Added via `add_cols_sql.py`
`dnalocation_rack`	TEXT	Added via `add_cols_sql.py`
`dnalocation_box`	TEXT	Added via `add_cols_sql.py`

GenomicLibraries

Column	Type	Notes
`library_id`	TEXT	**PRIMARY KEY**
`extraction_id`	TEXT	**FOREIGN KEY** → DNAExtractions
`lot_id`	TEXT	
`species`	TEXT	
`library_date`	TEXT	
`library_kit`	TEXT	
`qubit_dna_ng_ul`	REAL	

Column	Type	Notes
`input_mass_ng`	REAL	
`input_vol_ul`	REAL	
`eb_complement_ul`	REAL	
`frag_time_min`	INTEGER	
`cycles`	INTEGER	
`elution_ul`	REAL	
`qubit_lib_ng_ul`	REAL	
`size_selection`	TEXT	
`qubit_size_selection_ng_ul`	REAL	
`idt_primer_well`	TEXT	
`primer_name`	TEXT	
`i 5_index`	TEXT	
`i 7_index`	TEXT	
`bioanalyzer_avg_size`	INTEGER	
`insert_size`	INTEGER	
`concentration_nm_estimate`	REAL	
`data_target_gb`	REAL	
`librarylocation_id`	TEXT	Added via `add_cols_sql.py`
`librarylocation_shelf`	TEXT	Added via `add_cols_sql.py`
`librarylocation_rack`	TEXT	Added via `add_cols_sql.py`
`librarylocation_box`	TEXT	Added via `add_cols_sql.py`

Rebuilding the Database from Scratch

A complete database file including data from the Panama 2021 dataset and La Palma 2023 dataset is located at `cunha_invertebrate_specimens.db`

The scripts used to build the initial database are located in the folder `initial-building-scripts` and can be re-run to rebuild the database as it was upon project completion (5/1/2026)

> ⚠ **Important**
>
> The scripts must be run in a specific order. Running them out of order will either throw an error or produce an inconsistent database state.

To fully rebuild the database (for example, after making schema changes that require a clean slate):

```
python3 initial-building-scripts/panama_clean_load.py

python3 initial-building-scripts/la_palma_clean_load.py

python3 initial-building-scripts/add_cols_sql.py

python3 initial-building-scripts/fill_taxonomy.py
```

The expected terminal output for `panama_clean_load.py` should include something similar to this:

```
Loaded 33 rows into EventData
Loaded 1105 rows into SpecimenData
Loaded 782 rows into DNAExtractions
Loaded 11 rows into GenomicLibraries
```

The expected terminal output for `la_palma_clean_load.py` should include something similar to this:

```
Loaded 16 rows into EventData
Loaded 86 rows into SpecimenData
Loaded 34 rows into DNAExtractions
```

Each clean/load script follows the same internal structure:

1. Load raw CSVs into pandas DataFrames
2. Rename columns to match the database schema
3. Fill dataset-specific missing columns with `None`
4. Drop columns that are not being stored

5. Connect to the database
6. Add any new columns the dataset introduces via `ALTER TABLE`
7. Insert cleaned DataFrames into the database using `to_sql(if_exists='append')`

> ⚠ **Important**
>
> `panama_clean_load.py` includes an `os.remove()` call that deletes any existing `.db` file at the target path before creating a new one. **This means running it will permanently delete all data in the current database.**

Expected row counts after a clean build:

Table	After Panama	After La Palma
EventData	33	49
SpecimenData	1105	1191
DNAExtractions	782	816
GenomicLibraries	11	11

Adding a New Field Site Dataset

When a new field collection is added (e.g., a Japan expedition), the CSVs containing the data from that collection can be added via `verify_and_load.py` located in `database-scripts`.

Using verify_and_load.py

`verify_and_load.py` is an interactive script for safely adding new data to the existing database. It runs seven validation checks before inserting anything and clearly logs fatal errors and warnings. The script will never modify the database if a fatal error is found.

Running the script

```
python3 database-scripts/verify_and_load.py
```

The script will prompt you to select a table and then provide a path to your CSV.

The seven validation checks:

Check	Type	What it catches
1. Required columns present	Fatal	Missing primary or foreign key columns
2. No null primary keys	Fatal	Blank `lot_id`, `event_code`, etc.
3. No duplicate primary keys in the CSV	Fatal	Two rows with the same ID
4. No primary key conflicts with the database	Fatal	Trying to insert an ID that already exists
5. Foreign key references are valid	Fatal	`event_code` that doesn't exist in EventData
6. Unrecognized columns	Warning	Possible typos; data still loads
7. Missing optional columns	Warning	Will be inserted as NULL

Fatal errors block insertion entirely. **Warnings** prompt you to confirm before proceeding.

Orphan rows

When Check 5 fails, the invalid rows are automatically saved to a CSV at:

```
database-scripts/orphan_<TableName>_<fk_column>.csv
```

Review this file to determine whether the parent rows need to be loaded first, or whether the foreign key values are genuinely wrong in the source data.

Loading order matters!

Because of foreign key constraints, tables must be loaded in parent -> child order:

```
EventData → SpecimenData → DNAExtractions → GenomicLibraries
```

For example, if you try to load `SpecimenData` before the corresponding `EventData` rows exist, Check 5 will catch it and block insertion.

Generating Template CSVs

The `get_blank_csvs.py` script queries the live database schema and writes four blank CSV files with the correct column headers for each table. These are the files lab members should use when entering new data.

```
python3 database-scripts/get_blank_csvs.py
```

Output files are written to `template_CSVs/` with the prefix `TEMPLATE_`. Run this script any time the database schema changes so the templates stay current.

Schema Modification via Script

Adding additional columns

The `add_cols_sql.py` pattern is the general recommended approach for making schema changes programmatically. It uses `cursor.executescript()` to run multiple `ALTER TABLE` statements in one call:

```
import sqlite 3



conn = sqlite 3.connect("initial-building-
scripts/cunha_invertebrate_specimens.db")

cursor = conn.cursor()



cursor.executescript("""

ALTER TABLE SpecimenData ADD COLUMN new_column_name TEXT;

ALTER TABLE DNAExtractions ADD COLUMN another_column REAL;
```

```
""")



conn.commit()

conn.close()
```

The `add_missing_column()` helper function used in the clean/load scripts wraps `ALTER TABLE` in a `try/except` block so it silently skips columns that already exist.

```
def add_missing_column(cursor, table, column, col_type):

    try:

        cursor.execute(f"ALTER TABLE {table} ADD COLUMN {column} {col_type};")

        print(f"Added {column} to {table}")

    except sqlite 3.OperationalError:

        pass  # Column already exists; skip
```

Dropping a column

If you are on SQLite 3.35.0 or later, you can use the direct syntax:

```
ALTER TABLE SpecimenData DROP COLUMN column_to_remove;
```

Deleting Data

Deleting specific rows

```
DELETE FROM SpecimenData

WHERE lot_id = 'target_lot_id';
```

> **Note**
>
> With foreign keys ON, deleting a parent row (e.g., an `EventData` row) that still has children in `SpecimenData` will fail. You must delete or reassign the children first.

Deleting rows that match a condition

```
DELETE FROM DNAExtractions

WHERE extraction_date < '2020-01-01';
```

Deleting all rows from a table (preserve the table structure)

```
DELETE FROM GenomicLibraries;
```

Dropping an entire table

```
DROP TABLE IF EXISTS TableName;
```

This permanently removes the table and all its data. **Make sure to backup first as there is no undo for this!**

Advanced SQL Queries

These go beyond what the Beginner Guide covers. All of these can be run directly in the SQLite 3 shell or via a Python script. For further documentation regarding writing SQL queries, see: GitHub

Aggregate functions

Count, sum, average, min, and max across grouped rows:

```
-- Average DNA concentration per species
SELECT species,
       COUNT(*) AS extraction_count,
       ROUND(AVG(qubit_dna_ng_ul), 2) AS avg_qubit,
       MAX(qubit_dna_ng_ul) AS max_qubit
FROM SpecimenData
JOIN DNAExtractions ON SpecimenData.lot_id = DNAExtractions.lot_id
WHERE qubit_dna_ng_ul IS NOT NULL
GROUP BY species
ORDER BY avg_qubit DESC;
```

Filtering on aggregated results

`WHERE` filters individual rows; `HAVING` filters after grouping:

```
-- Only species with more than 10 specimen lots
SELECT species, COUNT(*) AS lot_count
```

```
FROM SpecimenData

GROUP BY species

HAVING lot_count > 10

ORDER BY lot_count DESC;
```

LEFT JOIN vs INNER JOIN

An `INNER JOIN` (default `JOIN`) only returns rows that have a match in both tables. A `LEFT JOIN` returns all rows from the left table, with NULLs where there is no match in the right table.

```
-- All specimens, including those with no DNA extraction (NULL columns
for unextracted)

SELECT SpecimenData.lot_id, species, extraction_id, qubit_dna_ng_ul

FROM SpecimenData

LEFT JOIN DNAExtractions ON SpecimenData.lot_id = DNAExtractions.lot_id;
```

Subqueries

```
-- Specimens whose species appears fewer than 3 times in the database

SELECT lot_id, species

FROM SpecimenData

WHERE species IN (

    SELECT species

    FROM SpecimenData
```

```
    GROUP BY species

    HAVING COUNT(*) < 3

);
```

CASE statements

Add a computed column based on conditional logic:

```
SELECT lot_id, species, qubit_dna_ng_ul,

    CASE

        WHEN qubit_dna_ng_ul >= 100 THEN 'High'

        WHEN qubit_dna_ng_ul >= 10  THEN 'Medium'

        WHEN qubit_dna_ng_ul > 0    THEN 'Low'

        ELSE 'Not extracted'

    END AS dna_quality

FROM SpecimenData

LEFT JOIN DNAExtractions ON SpecimenData.lot_id = DNAExtractions.lot_id;
```

Checking for NULL values

```
-- Specimens that have no DNA extraction record at all

SELECT lot_id, species

FROM SpecimenData
```

```
WHERE lot_id NOT IN (
    SELECT DISTINCT lot_id FROM DNAExtractions WHERE lot_id IS NOT NULL
);
```

Multi-table join across all four tables

```
SELECT
    EventData.country,
    EventData.date,
    SpecimenData.lot_id,
    SpecimenData.species,
    DNAExtractions.qubit_dna_ng_ul,
    GenomicLibraries.library_id
FROM SpecimenData
JOIN EventData ON SpecimenData.event_code = EventData.event_code
LEFT JOIN DNAExtractions ON SpecimenData.lot_id = DNAExtractions.lot_id
LEFT JOIN GenomicLibraries ON DNAExtractions.extraction_id =
GenomicLibraries.extraction_id
WHERE GenomicLibraries.library_id IS NOT NULL
ORDER BY EventData.date;
```

Exporting query results to CSV from the command line

```
sqlite 3 -header -csv initial-building-
scripts/cunha_invertebrate_specimens.db \

  "SELECT * FROM SpecimenData LIMIT 100;" > output.csv
```

Or inside the SQLite 3 shell:

```
.headers on

.mode csv

.output my_results.csv

SELECT species, COUNT(*) FROM SpecimenData GROUP BY species;

.output stdout
```

Database Maintenance

Making a backup

The simplest way to make a backup of the database is just a file copy. It's recommended before any major schema change or row edits to make a backup.

```
cp initial-building-scripts/cunha_invertebrate_specimens.db \

   initial-building-scripts/cunha_invertebrate_specimens_BACKUP_$(date
+%Y%m%d).db
```

Checking database integrity

```
sqlite 3 initial-building-scripts/cunha_invertebrate_specimens.db "PRAGMA
integrity_check;"
```

This code returns `ok` if the file is uncorrupted. Run it after any script that modifies the database if you suspect something went wrong.

Listing all current table schemas

```
sqlite 3 initial-building-scripts/cunha_invertebrate_specimens.db
".schema"
```

Checking row counts across all tables

```
SELECT 'EventData'       AS table_name, COUNT(*) AS row_count FROM
EventData

UNION ALL

SELECT 'SpecimenData',   COUNT(*) FROM SpecimenData

UNION ALL

SELECT 'DNAExtractions', COUNT(*) FROM DNAExtractions

UNION ALL

SELECT 'GenomicLibraries', COUNT(*) FROM GenomicLibraries;
```

This and the previous code may be useful for verifying the schema after making changes.